

UNIVERSITÉ DE SHERBROOKE · ÉCOLE DE GESTION

GIS805

Session 07 / 12

Soirée d'intégration : multi-star, drill-across et réel-vs-cible

Structuration et analyse multidimensionnelle

C

PROFESSEUR

**Prof. Carlos Denner
dos Santos**

Département des systèmes
d'information et méthodes
quantit...

DATE

4 juin 2026

LIEU

Longueuil

HORAIRE

19 h 00 – 22 h 00

Votre séance, en bref

Voici comment se déroule votre soirée — 3 temps forts.

20 min

Multi-star theory + bus matrix

45 min

Sprint 1 : integration night

40 min

Sprint 2 : actual-vs-budget + board report



*Le board peut-il voir ventes, retours,
inventaire et budget dans une seule
vue sans mentir ?*

— Le CEO de NexaMart Group, séance 6

Ce soir : vos étoiles deviennent un entrepôt

Soiree d'integration NexaMart

- Jusqu'ici, vous avez construit chaque étoile independamment
- **Mais le CEO veut une vue consolidee** : ventes vs retours vs budget
- Cela nécessite des dimensions conformes et un drill-across correct
- **Erreur fatale** : joindre deux tables de faits directement

Ce que vous avez construit jusqu'ici

Quatre étoiles indépendantes — chacune avec son propre grain

Table de faits	Grain	Dimensions clés	Particularité
fact_sales	sale_line_id	customer, product, store, channel	S01-S04
fact_returns	return_id	product, store, customer (SCD2)	S06 — NEW
fact_inventory_snapshot	product × store × semaine	product, store	S06 — mesure semi-additive
fact_budget	category × store × mois	store seulement	S06 — category = VARCHAR, PAS de product_key

❗ Prenez 2 minutes pour lire chaque grain. Le grain de fact_budget sera le point d'achoppement ce soir — Exercice 0 vous le fera découvrir.

Quelles dimensions sont présentes dans chaque table ?

La bus matrix en preview — vous la formaliserez en Exercice 1

Dimension	fact_sales	fact_returns	fact_inventory_snapshot	fact_budget
dim_date (date_key)	✓	✓	✓	✓
dim_product (product_key)	✓	✓	✓	category VARCHAR
dim_store (store_key)	✓	✓	✓	✓
dim_customer (customer_key)	✓	✓	—	—
dim_channel (channel_key)	✓	—	—	—

❗ fact_budget n'a intentionnellement ni product_key ni customer_key. Les budgets sont planifiés par catégorie — pas par produit. Exercice 0 vous fera découvrir pourquoi.

01 La bus matrix

L'outil d'integration le plus important de l'entrepôt

Qu'est-ce que la bus matrix ?

Le plan d'architecture de votre entrepôt

- Colonnes = tables de faits (fact_sales, fact_returns, fact_inventory, fact_budget)
- Lignes = dimensions (dim_date, dim_product, dim_store, dim_customer)
- Case cochée = la dimension est conforme et partagée entre les deux tables
- Case vide = décision à documenter (manquante ou intentionnellement exclue)
- La bus matrix est le document le plus important pour l'intégration

Exemple : la bus matrix NexaMart remplie

Voici à quoi elle ressemble quand elle est complétée

Dimension	fact_sales	fact_returns	fact_inventory_snapshot	fact_budget
dim_date (date_key)	✓	✓	✓	✓
dim_product (product_key)	✓	✓	✓	✓
dim_store (store_key)	✓	✓	✓	—
dim_customer (customer_key)	✓	✓	—	—
dim_promo (promo_key)	✓	—	—	—

❶ ✓ = dimension conforme partagée. — = pas applicable à ce processus. Remarquez : dim_date et dim_product sont conformes partout — ce sont vos piliers d'intégration.

Le danger du produit cartésien

Pourquoi on ne joint JAMAIS deux tables de faits

- fact_sales a 10 000 lignes pour le produit X en janvier
- fact_returns a 500 lignes pour le même produit en janvier
- **JOIN direct** : $10\,000 \times 500 = 5\,000\,000$ lignes -- produit cartésien
- Le revenue est multiplié par 500, les retours par 10 000
- **Solution** : le drill-across via les dimensions conformes

FAUX vs CORRECT : le JOIN mortel et le drill-across

Voyez la difference avant d'ecrire une seule ligne

- [FAUX] JOIN direct entre deux tables de faits
- `SELECT p.category, SUM(s.line_total) AS rev, SUM(r.refund_amount) AS ref`
- `FROM fact_sales s`
 - `JOIN fact_returns r USING (product_key) -- 10 000 × 500 = 5 millions de lignes`
 - `JOIN dim_product p USING (product_key)`
- **GROUP BY 1; -- FAUX** : rev gonfle x500, ref gonfle x10000
- [CORRECT] Deux CTEs separees, meme grain, FULL OUTER JOIN sur les resultats
- `WITH sales_agg AS (SELECT ... GROUP BY category, mois),`
 - `returns_agg AS (SELECT ... GROUP BY category, mois)`

❗ Si deux tables de faits apparaissent dans le meme FROM sans CTE : c'est un produit cartesien garanti.

FAUX vs CORRECT : le JOIN mortel et le drill-across (suite)

- `SELECT ... FROM sales_agg s`
 - `FULL OUTER JOIN returns_agg r ON s.category=r.category AND s.mois=r.mois;`

02 Le drill-across

Comparer des métriques de tables différentes sans duplication

Comment fonctionne le drill-across

Deux requêtes, même dimension, même grain

- **Étape 1** : agréger fact_sales par (product, month) → revenue_par_produit
- **Étape 2** : agréger fact_returns par (product, month) → retours_par_produit
- **Étape 3** : joindre les deux resultats sur (product, month)
- **Resultat** : revenue ET retours côte à côte, sans produit cartésien
- **Le grain commun est la clé** : les deux requêtes doivent agréger au même niveau

Drill-across : le squelette CTE complet

Référence — essayez d'abord en Exercice 2 avant de consulter ce slide

- WITH
 - sales_agg AS (
 - SELECT p.category,
 - DATE_TRUNC('month', f.order_date) AS mois,
 - SUM(f.line_total) AS revenue,
 - SUM(f.quantity) AS units_sold
 - FROM fact_sales f JOIN dim_product p USING (product_key)
 - GROUP BY 1, 2),
 - returns_agg AS (
 - SELECT p.category,
 - DATE_TRUNC('month', r.return_date) AS mois,
 - SUM(r.refund_amount) AS total_refunds,
 - SUM(r.return_quantity) AS units_returned
 - FROM fact_returns r JOIN dim_product p USING (product_key)
 - GROUP BY 1, 2)
 - SELECT COALESCE(s.category, r.category) AS category,

fact_budget n'a pas de product_key — et c'est intentionnel

Découvrez le design de cette table avant de la joindre

- Exécutez ceci et observez les colonnes :
- DESCRIBE fact_budget;
- **Colonnes** : budget_id | budget_month | category | store_key | target_revenue | target_units
- Pas de product_key. Pas de customer_key.
- Les décisions budgétaires sont prises au niveau catégorie × magasin × mois — pas au niveau produit.
- **Conséquence** : on ne peut pas joindre fact_budget à fact_sales via product_key.

❗ Exercice 3 vous demandera de résoudre ce défi vous-même. Le slide suivant donne la clé conceptuelle.

fact_budget n'a pas de product_key — et c'est intentionnel (suite)

- **Question** : comment joindre ces deux tables malgré cette différence ?

La clé : descendre au grain du budget via dim_product

Le pont conceptuel — avant le SQL

- `fact_sales.product_key` → (via `dim_product`) → `dim_product.category`
 - ↕
 - `fact_budget.category` (VARCHAR brut)
- **Étape 1** : agréger `fact_sales` par `category` × `store_key` × `mois` via `dim_product`
- **Étape 2** : FULL OUTER JOIN sur le grain commun (`category`, `store_key`, `mois`)
- **Ce n'est pas un bug — c'est le design voulu** : `category` est une dimension dégénérée dans `fact_budget`.
- `dim_product` est le pont qui rend ce drill-across possible.

❗ Retenez : chaque fois que `fact_budget` apparaît dans un FROM, `dim_product` doit accompagner `fact_sales`.

SQL : vue réel-vs-budget

Référence — essayez d'abord en Exercice 3 avant de consulter ce slide

- WITH actual AS (
 - SELECT p.category, f.store_key, DATE_TRUNC('month', f.order_date) AS mois,
 - SUM(f.line_total) AS actual_revenue
 - FROM fact_sales f JOIN dim_product p USING (product_key) GROUP BY 1, 2, 3)
- SELECT COALESCE(a.category, b.category) AS category,
 - COALESCE(a.store_key, b.store_key) AS store_key,
 - COALESCE(a.mois, b.budget_month) AS mois,
 - a.actual_revenue, b.target_revenue,
 - (a.actual_revenue - b.target_revenue) AS variance,
 - ROUND(100.0*(a.actual_revenue - b.target_revenue)
 - / NULLIF(b.target_revenue, 0), 1) AS pct_variance
- FROM actual a FULL OUTER JOIN fact_budget b
 - ON a.category = b.category AND a.store_key = b.store_key AND a.mois = b.budget_month
- ORDER BY mois, category;
- variance = réel - budget. pct_variance = % d'écart. NULLIF évite la division par zéro.

Votre copilote cette semaine

Comment travailler avec votre assistant IA pour le drill-across

✓ PROMPTS QUI MARCHENT

- Pourquoi ne peut-on JAMAIS joindre deux tables de faits directement ?
- Ecris un drill-across : agrege ventes et retours au même grain, puis joins.
- Genere une vue actual-vs-budget avec variance.

✗ PROMPTS À REJETER

- l'assistant joint fact_sales a fact_returns directement (produit cartésien)
- les totaux drill-across ne reconcilent pas avec les totaux individuels

03 Sprint 1 : integration night

Charger 4 tables de faits et vérifier la conformité

Votre mission

45 minutes pour intégrer vos étoiles

- 1. Chargez les 4 tables de faits dans DuckDB
- 2. Vérifiez que dim_date et dim_product sont conformes entre toutes les tables
- 3. Ecrivez un drill-across entre fact_sales et fact_returns
- 4. Documentez votre bus matrix dans docs/bus-matrix.md

❗ Erreur fréquente : JOIN fact_sales TO fact_returns directement. Arrêtez-vous si vous voyez ça.

Sprint 1 : votre mission

Fichier cible : `sql/integration/s07-drill-across.sql`

- **Étape 0 (Exercice 0)** : DESCRIBE les 4 tables — répondez aux 3 questions dans vos commentaires.
- **Étape 1 (piège)** : essayez le JOIN direct `fact_sales JOIN fact_returns USING (product_key)`.
 - → Comparez `SELECT COUNT(*)` obtenu vs les deux tables séparées. Notez le ratio d'inflation.
- **Étape 2** : construisez deux CTEs — ventes et retours — au même grain (`category × mois`).
 - → Décrivez le grain dans un commentaire. Pas de skeleton fourni : construisez-le.
- **Étape 3** : FULL OUTER JOIN sur (`category, mois`). Jamais INNER JOIN.
- **Réconciliation obligatoire** : `SUM(revenue CTE) = SUM(line_total FROM fact_sales)`.
 - → Si non : GROUP BY incomplet. Ajoutez mois.
- `git add -A && git commit -m 'S07 sprint1 drill-across fonctionnel'`

❗ Les slides de théorie restent disponibles comme référence. Mais construisez d'abord — consultez ensuite.

Sprint 1 : votre mission (suite)

- **Étape 4** : bus matrix dans docs/bus-matrix.md (Exercice 1).

04 Sprint 2 : réel vs budget

Le rapport que le CFO attend

Sprint 2 : votre mission

Fichier cible : `sql/integration/s07-actual-vs-budget.sql`

- **Étape 0 (Exercice 3, anatomie)** : `DESCRIBE fact_budget` — répondez dans un bloc `-- OBSERVATIONS` :
 - → Y a-t-il un `product_key` ? Quel est le grain exact ?
- **Étape 1 (piège)** : tentez `JOIN fact_sales JOIN fact_budget ON store_key` sans `dim_product`.
 - → Qu'arrive-t-il aux catégories ? Notez le résultat dans un commentaire.
- **Étape 2** : CTE `actual` — agréger `fact_sales` via `dim_product` → `category × store_key × mois`.
 - → Le pont : `dim_product.category = fact_budget.category`.
- **Étape 3** : `FULL OUTER JOIN fact_budget` sur le grain commun (`category`, `store_key`, `budget_month`).
- **Étape 4** : ajoutez `variance` et `pct_variance`. Utilisez `NULLIF` pour éviter la division par zéro.
- **Réconciliation obligatoire** : `SUM(actual_revenue) = SUM(fact_sales.line_total)`.

❗ Le slide de référence « SQL : vue réel-vs-budget » est disponible si vous êtes bloqué après avoir essayé.

Sprint 2 : votre mission (suite)

- `git add -A && git commit -m 'S07 sprint2 actual-vs-budget + reconciliation'`

Livrable final

Vue consolidee + board brief

- **Vue SQL** : actual_revenue vs budget par catégorie x magasin x mois
- **Reconciliation** : prouver que les totaux n'incluent pas de duplication
- **Board brief** : répondre a la question du CEO avec evidence multi-table

Soumission et evaluation

Ce qui est attendu avant S08

- **Fichiers** : answers/S07_executive_brief.md + docs/bus-matrix.md
- **SQL** : sql/integration/s07-drill-across.sql + sql/integration/s07-actual-vs-budget.sql
- **Evaluation** : modèle (40%), validation (25%), justification exécutive (20%)
- Trace de processus (10%), reproductibilite (5%)
- **Commitez** : git add -A && git commit -m "S07 integration night" && git push
- Consultez docs/verify-before-pushing.md avant de pousser

Où en sommes-nous ?

Votre parcours en 14 séances

- S01-S04 : Étoile, grain, SCD, dims spéciales [FAIT]
- S05 : Révision + prérequis [FAIT]
- S06 : Examen intra 1 [FAIT]
- >> **S07** : Integration multi-star, bus matrix, drill-across [AUJOURD'HUI]
 - S08 : Dates role-playing, hiérarchies, NULLs
 - S09 : 4 types de tables de faits
 - S10 : Examen intra 2 (S06-S09)
 - S11-S14 : Documentation, défense, au-dela, final

A retenir de ce soir

- 1 JAMAIS joindre deux tables de faits directement -- produit cartésien garanti.
- 2 Drill-across = deux requêtes au même grain, jointes sur les dimensions conformes.
- 3 La bus matrix est le document d'architecture central de l'entrepôt.
- 4 Le grain commun est la clé du réel-vs-budget.
- 5 La semaine prochaine : dates role-playing, hiérarchies et NULLs (S08).